

THESIS PROPOSAL

Contrastive Embedding-Based Detection of Prompt Injection Attacks in LLM-Integrated Applications

*A Lightweight, Explainable and Empirically Validated Defense Framework
for LLM Agent Security*

Submitted by: Md. Mehedi Hasan and Team

Supervisor: Prof. Dr. Md. Abdul Based
Head, Department of Computer Science & Engineering
Dhaka International University

Abstract

Prompt injection and jailbreak attacks are a growing threat to LLM-integrated applications, exploiting the inability of current models to reliably separate developer instructions, user input and untrusted third-party content. This proposal presents a project-based, empirically driven thesis to design, train and deploy a lightweight contrastive embedding classifier (DistilBERT encoder with an InfoNCE objective) as an explainable, real-time pre-filter against such attacks. The study is structured around six pre-registered research questions covering detection performance, the marginal contribution of contrastive training, cross-attack-family generalization, adversarial robustness, explanation quality and real-world deployment behavior inside an active LLM agent pipeline. The methodology combines public benchmark data (JailbreakBench, HarmBench, AdvBench-derived sets) with self-generated adversarial examples, a held-out attack family for genuine generalization testing and a live shadow-mode deployment with internal red-teaming for out-of-distribution validation. Statistical rigor is enforced through paired significance testing, bootstrapped confidence intervals and multiple-comparison correction. The expected contribution is both academic (a reusable, generalizable methodology for lightweight LLM-security detectors) and applied (a deployable safety component), with a secondary socio-economic contribution around lowering the cost barrier for LLM-safety adoption in resource-constrained settings.

1 Introduction

Large Language Models (LLMs) have moved from standalone chat interfaces into agentic, tool-using applications [16] that read external content, call APIs and take actions on a user's behalf. This shift has introduced a rapidly growing attack surface prompt injection [1, 3], where adversarial text embedded in user input or in third-party content the model consumes overrides the system's intended instructions. Related jailbreak attacks use adversarial phrasing, role-play framing or encoded payloads to bypass a model's safety alignment directly [4, 6].

This thesis proposes a project-based, experimental study of prompt injection and jailbreak detection using contrastive sentence embeddings [14]. The goal is to build a lightweight,

explainable pre-filter that sits between raw user input and an LLM agent, flagging adversarial input before it reaches the underlying model. Unlike prior work that evaluates almost exclusively on static, in-distribution benchmarks, this thesis is designed around a deployment-grounded evaluation. The classifier is integrated into an LLM agent pipeline and evaluated on live traffic, in addition to standard benchmark metrics.

2 Problem Statement

Let x denote a raw user input to an LLM-integrated application, and let $f(x)$ denote the application’s intended behavior under a fixed system instruction s . An input x is classified as a prompt injection or jailbreak attack if it causes the deployed model to deviate from $f(x)$ in a way that violates the system’s intended policy, either by overriding s directly (direct injection) [1] or by smuggling adversarial instructions through retrieved third-party content c that the model processes alongside x (indirect injection) [2].

Formally, the thesis targets a binary detection function:

$$D : x \rightarrow \{0, 1\}, \quad D(x) = 1 \text{ if } x \text{ (or } c \text{) contains adversarial intent to override } s, \text{ else } 0$$

The research problem is to learn D such that it:

- (i) achieves high precision and recall against known and unseen attack families,
- (ii) remains robust to adversarial paraphrasing of x ,
- (iii) executes within real-time latency budgets suitable for a pre-filter position in a request pipeline,
- (iv) produces a human-interpretable rationale for every positive decision.

Existing defenses fall short on at least one of these four criteria: rule-based filters fail (i) and (ii); large commercial guardrail models [9] satisfy (i) but often violate (iii) on cost/latency grounds and (iv) on interpretability grounds; and academic classifiers evaluated only on static benchmarks [7, 8] rarely demonstrate (ii) or provide direct evidence for (iii)–(iv) under live deployment.

3 Why This Research

- Prompt injection has been recognized as a top-tier security risk for LLM-integrated applications (OWASP Top 10 for LLM Applications [19]), and published research volume on the topic has grown sharply since 2023, indicating both urgency and an active, receptive research community.
- It produces a reusable, deployable artifact, giving the thesis a genuine engineering and safety contribution alongside its academic one.
- It aligns with current directions in AI safety and interpretability research.

4 Threat Model

To keep the empirical scope well defined, the thesis adopts the following threat model, consistent with prior LLM-security literature [1, 2, 3]:

- **Attacker capability:** the attacker can submit arbitrary natural-language input directly to the application (direct injection), or control third-party content that the application later retrieves and feeds to the model (indirect injection).

- **Attacker goal:** goal hijacking (forcing the model to perform an unintended action or produce disallowed content) or prompt leaking (extracting hidden system instructions).
- **Attacker knowledge:** black-box with respect to the detector (the attacker does not have gradient access to the classifier), but white-box with respect to publicly known jailbreak techniques and templates.
- **Defender capability:** the defender controls the pre-filter model and its training data, and can log and monitor flagged/unflagged traffic in shadow mode before enforcement.
- **Out of scope:** attacks that require model weight access, training-time data poisoning of the target LLM, and physical/side-channel attacks are excluded from this thesis; these are noted as directions for future work in Section 15.

5 Related Work

5.1 Prompt Injection and Jailbreaking

Prompt injection was first formally characterized as a distinct threat class by Perez and Ribeiro [1] and Liu et al. [3], describing how natural-language instructions embedded in user input could override a system’s intended directives. This was later extended by Greshake et al. [2] to indirect prompt injection, where adversarial instructions are embedded in third-party content that an LLM agent retrieves and processes, substantially broadening the practical attack surface for tool-using agents. Gradient-based optimization attacks such as GCG [4] demonstrated that adversarial suffixes could be found automatically and transferred across aligned models, while empirical studies of in-the-wild jailbreaks [5, 6] highlighted role-play and encoding strategies, motivating the need for detectors that generalize beyond hand-crafted attack templates.

5.2 Benchmarks and Evaluation Frameworks

HarmBench [7] provides a standardized evaluation framework for automated red-teaming and robust refusal testing, while JailbreakBench [8] offers a curated set of harmful behaviors alongside a public leaderboard, explicitly built to unify inconsistent prior evaluation practices across the field. AdvBench-derived attack sets [4] are widely used as a common seed corpus across this literature, though prior surveys note that most released benchmarks lack systematic cross-family generalization testing, a gap this thesis directly addresses in Section 9.4.

5.3 Defense Mechanisms

Meta’s Llama Guard [9] introduced an LLM-based input-output safeguard trained specifically to classify conversational turns as safe or unsafe, representing the commercial-guardrail category benchmarked in this thesis. This class of defense achieves strong coverage but at the cost of running a second, often large, LLM as a judge, which is expensive and typically opaque. Lighter-weight approaches, including perplexity-based anomaly detection [10, 11], offer speed but weaker robustness to fluent, well-formed adversarial prompts. This thesis targets the underexplored middle ground: a lightweight, self-hosted encoder-based classifier with built-in explainability, rather than either a heavyweight LLM judge or a brittle rule-based filter.

5.4 Representation Learning Foundations

The technical building blocks for the proposed detector are grounded in established work: Distil-BERT [12] provides a distilled, efficient encoder architecture suitable for real-time inference; the InfoNCE contrastive objective [13], introduced in representation learning research on contrastive

predictive coding, underlies the embedding-separation strategy used here and in contrastive sentence embedding frameworks like SimCSE [14]; and SHAP [15] provides a unified, theoretically grounded approach to feature attribution, applied here to surface which tokens drive a detection decision.

5.5 Research Gap

Across this literature, three gaps recur:

1. Evaluation is dominated by in-distribution benchmark testing, with limited attention to cross-attack-family generalization.
2. Defenses are either heavyweight (LLM-judge guardrails) or brittle (rule-based filters), with few lightweight, explainable middle-ground detectors validated empirically.
3. Almost no published work validates a detector against live, real-world deployment traffic rather than static test sets.

This thesis is scoped specifically to address all three gaps within a single, coherent empirical study.

6 Theoretical Framework

The proposed detector rests on the hypothesis that benign and adversarial LLM inputs occupy separable regions of a learned embedding space when that space is explicitly optimized for this separation, rather than inherited incidentally from a generic language-model pretraining objective. This motivates the use of a contrastive objective rather than a standard cross-entropy classifier alone.

Given an anchor input x_i , a positive example x_i^+ from the same class and a set of negative examples $\{x_j^-\}$ from the opposite class, the InfoNCE loss [13] used for encoder training is:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_j \exp(\text{sim}(z_i, z_j)/\tau)}$$

where $z_i = \text{Enc}(x_i)$ is the DistilBERT-encoded representation [12], $\text{sim}(z_i, z_j)$ is cosine similarity and τ is a temperature hyperparameter. This objective directly shapes the embedding geometry, in contrast to cross-entropy training, which optimizes only the final decision boundary and does not explicitly enforce intra-class compactness or inter-class separation in the representation space. The theoretical expectation, tested empirically via RQ2, is that this explicit geometric structuring improves generalization to unseen attack families (RQ3) because it encourages the encoder to learn transferable notions of 'adversarial intent' rather than memorizing surface patterns specific to the training attack families.

7 Research Questions

- RQ1:** Can a contrastive embedding-based classifier (DistilBERT encoder [12] trained with an InfoNCE objective [13]) detect prompt injection and jailbreak attempts with precision and recall competitive with heavier baseline classifiers?
- RQ2:** Does the contrastive training objective meaningfully outperform a standard cross-entropy fine-tuned classifier on this task, and by how much (effect size, not just significance)?

- RQ3:** Does a detector trained on one family of attacks (e.g. role-play jailbreaks [5]) generalize to a structurally different, unseen attack family (e.g. encoded-payload injection) held out entirely from training?
- RQ4:** How much does detection performance degrade under adversarial paraphrasing and obfuscation of known attacks, and does this degradation differ between the contrastive and cross-entropy models?
- RQ5:** Can SHAP-based token attribution [15] over the contrastive encoder produce explanations that are consistent across similar inputs and rated as useful by human evaluators for auditing false positives and false negatives?
- RQ6:** Is the resulting classifier fast enough to serve as a real-time pre-filter and what is the observed false-positive rate and latency distribution when deployed in shadow mode on live traffic inside an actual LLM agent pipeline?

8 Objectives

- Construct a labeled dataset of 15,000–30,000 examples combining public prompt injection and jailbreak benchmarks [4, 7, 8] with self-generated adversarial variants, alongside a large and diverse benign query set, with one attack family fully withheld for generalization testing.
- Design and train a DistilBERT encoder [12] with an InfoNCE contrastive objective [13] to separate benign and adversarial queries in embedding space, with a matched cross-entropy counterpart for controlled comparison.
- Benchmark the proposed detector against rule-based, cross-entropy, perplexity-based [10, 11], and commercial guardrail (Llama Guard [9]) baselines under identical evaluation protocol.
- Evaluate cross-attack-family generalization and adversarial robustness under paraphrasing and obfuscation, with statistical significance testing at each stage.
- Integrate SHAP-based explainability [15] so that every flagged input is accompanied by a token-level rationale, and validate explanation quality with a small human evaluation.
- Deploy the detector as a live, shadow-mode pre-filter inside the SocialAI pipeline and evaluate real-world false-positive rate and latency before any enforcement decision.
- Conduct an internal red-teaming exercise to generate genuinely novel attacks for a true out-of-distribution evaluation, documented under the ethical protocol in Section 13.

9 Proposed Methodology

9.1 Dataset Construction

- **Public sources:** established prompt injection and jailbreak benchmarks (HuggingFace prompt-injection datasets, JailbreakBench [8], AdvBench [4]/HarmBench [7]-derived sets) as the adversarial-class seed, all under research-permissive licensing.
- **Synthetic augmentation:** SocialAI’s LLM pipeline generates additional injection variants, including role-play jailbreaks [5], instruction-override phrasing, encoded/obfuscated payloads, and multi-turn escalation attacks.

- **Benign class:** diverse, large-scale normal user queries drawn from public chatbot and assistant Q&A datasets, to avoid the classifier simply learning to flag unusual phrasing.
- **Stratification:** the full dataset is stratified by attack family and split into training (70%), validation (15%), and held-out test (15%) partitions, with one entire attack family withheld from training to support the generalization test in RQ3.
- **Annotation quality control:** a random 5% sample of self-generated adversarial examples is manually reviewed against the pre-registered attack taxonomy to check label validity before training.

9.2 Model Architecture

- **Encoder:** DistilBERT [12], chosen for its balance of accuracy and inference speed, consistent with prior production use in Tactic Detector.
- **Training objective:** InfoNCE contrastive loss [13] (Section 6), pulling benign queries together and separating them from injection/jailbreak queries in embedding space, with in-batch hard-negative mining.
- **Classification head:** a lightweight MLP or cosine-similarity threshold applied on top of the contrastive embedding space, tuned on the validation set for a target operating point balancing precision and recall.
- **Explainability layer:** SHAP attribution [15] computed over encoder token contributions, surfacing which phrases drove a given classification.

9.3 Baselines

Table 1: Baseline Models and Benchmarking Purpose

Baseline	Purpose
Rule-based / regex filters	Naive baseline demonstrating the limits of pattern match against paraphrased attacks.
Vanilla fine-tuned DistilBERT (cross-entropy) [12]	Isolates the specific contribution of the InfoNCE contrastive objective (directly answers RQ2).
Perplexity-based detection [10, 11]	A known, simple LLM-security baseline for anomalous input detection.
Llama Guard (commercial guardrail) [9]	Strong external reference point against an industry-grade safeguard model.

9.4 Experimental Design

- **Detection performance:** precision, recall, F1, and AUROC on the held-out test set, computed per attack family and in aggregate.
- **Generalization test:** train on all attack families except one, evaluate on the withheld family only, isolating true generalization from memorization.
- **Adversarial robustness:** paraphrased and obfuscated variants of known attacks generated at increasing perturbation strength, measuring detection-rate degradation as a curve rather than a single number.

- **Latency benchmark:** end-to-end inference time under real-time constraints, measured on representative hardware.
- **Ablation study:** with vs. without contrastive loss; with vs. without synthetic data augmentation; with vs. without SHAP-guided hard-negative mining; each run across multiple random seeds.
- **Deployment study:** shadow-mode integration into SocialAI, logging decisions without blocking, to measure real-world false-positive rate and latency before any enforcement mode is enabled.
- **Red-teaming:** an internal exercise generating novel attacks not present in any training or benchmark data, used as a final out-of-distribution check, run under the ethical protocol in Section 13.

10 Statistical Analysis Plan

- All model comparisons (contrastive vs. cross-entropy, proposed model vs. each baseline) are evaluated with paired significance tests across cross-validation folds or bootstrap resamples, not single point-estimate comparisons.
- Confidence intervals for precision, recall, F1, and AUROC are computed via 5,000-resample non-parametric bootstrapping [17], consistent with rigorous empirical evaluation standards.
- Where multiple attack families or multiple ablation conditions are compared simultaneously, Benjamini-Hochberg false discovery rate correction [18] is applied to control for multiple-comparison inflation.
- Effect sizes (e.g. Cohen’s d or matched odds ratios) are reported alongside p -values for every hypothesis test, so that statistically significant but practically negligible differences are not overstated.
- All experiments are run across at least three random seeds, with variance reported, to distinguish genuine effects from training-noise artifacts.
- Any null or negative result is reported alongside positive findings rather than omitted.

11 Expected Results Outcomes & Impacts On Socio-Economics

As this is a proposal document, no experimental results exist yet. Table 2 outlines the expected result structure and reporting plan for the completed thesis, so that outcomes can be evaluated directly against the research questions in Section 7 once experiments are run.

Beyond the direct research contributions, this thesis is expected to produce outcomes at three levels: technical, organizational, and broader socio-economic.

- **Technical:** a validated, reusable, explainable pre-filter artifact, integrated into an active LLM agent pipeline rather than left as a standalone research prototype.
- **Organizational:** a documented methodology for dataset construction, adversarial evaluation and deployment monitoring that can be reused for future LLM-security work in the research community.

LLM-integrated applications are increasingly deployed in customer service, education, health-care triage and financial advisory contexts across South Asia and other emerging markets, where organizations often cannot afford continuous calls to large commercial guardrail models due

Table 2: Planned Results and Mapping to Research Questions

Research Question	Planned Result Type
RQ1 – detection performance	Comparative table of precision, recall, F1, AUROC with bootstrap CIs [17] across all baselines and the proposed model.
RQ2 – contrastive vs. cross-entropy	Paired significance test with effect size, isolating the InfoNCE objective’s contribution.
RQ3 – cross-family generalization	Confusion matrix and F1 breakdown per attack family, including the fully withheld family.
RQ4 – adversarial robustness	Detection-rate degradation curve under increasing paraphrase/obfuscation strength, contrastive vs. cross-entropy.
RQ5 – explainability	Qualitative SHAP [15] token-attribution examples plus a small human-rated usefulness and consistency check.
RQ6 – deployment	Real-world false-positive rate and latency distribution from shadow-mode SocialAI logs.

to cost and infrastructure constraints. A lightweight, self-hostable detector directly addresses this economic gap: it lowers the cost barrier for smaller organizations and startups to deploy LLM-integrated products safely, without depending on expensive third-party moderation APIs.

- **Cost reduction:** a smaller, self-hosted classifier reduces the operating cost of input moderation compared to routing every request through a large commercial guardrail model, which matters disproportionately for resource-constrained organizations and student-built platforms such as SocialAI.
- **Trust and adoption:** reliable, explainable safety filtering lowers the risk of reputational or financial harm from a successful prompt injection attack, a material concern for any organization adopting LLM agents commercially.
- **Workforce and skills:** the project builds transferable expertise in LLM security, a fast-growing and currently under-supplied skill area, both for the student researchers involved and for the broader DIU CPC community through knowledge-sharing sessions.
- **Equity of access:** because the approach avoids dependence on expensive proprietary APIs, it is more accessible to researchers and small teams in lower-resource settings, aligning with the socio-economic accessibility angle present in the team’s broader research portfolio.

12 Ethical Considerations

- **Dual-use risk:** synthetic adversarial examples generated via SocialAI are used strictly for training and evaluation of the defense system, are not released publicly in raw form, and are handled under the same access controls as other sensitive research data.
- **Red-teaming scope:** the internal red-teaming exercise (Section 9.4) is limited to team members, is scoped to input-classification bypass only, and does not target real third-party systems or production infrastructure outside SocialAI’s own sandboxed environment.
- **Deployment safety:** the live evaluation is run in shadow mode only, meaning flagged input is logged but never blocked or acted upon during the evaluation period, so no real user is affected by detector errors during the study.

- **Data handling:** all live-traffic logs used for the deployment study are anonymized before analysis, and only aggregate metrics (false-positive rate, latency) are reported in the thesis, consistent with the university’s research data handling guidelines.
- **Responsible disclosure:** any previously undocumented attack pattern discovered during red-teaming that could generalize to third-party production systems will be reported through appropriate responsible-disclosure channels rather than published in raw form.

13 Limitations

- The dataset, while stratified across known attack families, cannot guarantee coverage of all future or unknown attack strategies; generalization results should be read as evidence of relative robustness, not a guarantee of complete coverage.
- Synthetic adversarial examples generated via SocialAI may carry stylistic artifacts of the generating LLM, which could bias the classifier toward detecting machine-generated phrasing specifically; this is partially addressed by RQ3’s cross-family test but remains a limitation to be discussed explicitly in the final manuscript.
- The shadow-mode deployment study is scoped to SocialAI’s own traffic, which may not represent the full diversity of attack patterns seen by larger, more heavily targeted production LLM applications.
- As with any single-team academic evaluation, results should be interpreted alongside, not as a replacement for, independent third-party red-teaming before any production enforcement decision is made.

14 Proposed Timeline

Table 3: Proposed Project Timeline

Phase	Milestone	Target
Phase 1	Dataset construction: public sources aggregated, SocialAI synthetic generation pipeline built and run, held-out attack family selected.	Weeks 1–3
Phase 2	Model training: DistilBERT + InfoNCE encoder, cross-entropy counterpart, and all baseline models trained.	Weeks 3–4
Phase 3	Core evaluation: precision/recall/F1/AUROC, generalization test, statistical analysis per Section 10.	Week 5
Phase 4	Robustness and efficiency: adversarial perturbation tests, latency benchmarking, ablations across seeds.	Weeks 5–6
Phase 5	Deployment integration into SocialAI as a shadow-mode pre-filter; false-positive and latency monitoring.	Weeks 6–7
Phase 6	Internal red-teaming exercise (under ethical protocol) and human evaluation of SHAP explanations.	Weeks 7–8
Phase 7	Manuscript drafting, statistical writeup, and figures.	Weeks 8–10
Phase 8	Internal defense rehearsal and submission.	Week 11

15 Resource Requirements

- **Compute:** GPU access sufficient for DistilBERT fine-tuning (a single mid-range GPU or Colab-class instance is sufficient given the model scale).
- **Software:** HuggingFace Transformers, SHAP library [15], standard Python data-science stack; all open-source and already in use in prior team projects.
- **Data access:** public benchmark datasets [4, 7, 8] under research-permissive licenses; no paid data acquisition required.
- **Infrastructure:** staging access to the SocialAI pipeline for shadow-mode deployment, coordinated with the platform’s existing maintainers on the team.

16 Conclusion

This thesis proposes an experimental, deployment-grounded, and statistically rigorous study of prompt injection and jailbreak detection using contrastive sentence embeddings [14], directly building on the team’s existing expertise in DistilBERT-based classification [12] and InfoNCE contrastive training [13]. Rather than a purely benchmark-driven exercise, the study is designed to test generalization across attack families, adversarial robustness, real-time efficiency, explanation quality [15], and real-world deployment behavior inside an active LLM agent pipeline, with an explicit statistical analysis plan [17, 18] and ethical protocol governing every experimental stage. The resulting work is intended to serve as both a defensible, PhD-grade academic thesis and a reusable safety component for the team’s own systems, with clear relevance to the broader socio-economic goal of making LLM safety accessible to resource-constrained organizations.

17 References

References

- [1] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” *arXiv preprint arXiv:2211.09527*, 2022.
- [2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” *arXiv preprint arXiv:2302.12173*, 2023.
- [3] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, K. Wang, and Y. Liu, “Prompt injection attack against LLM-integrated applications,” *arXiv preprint arXiv:2306.05499*, 2023.
- [4] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” *arXiv preprint arXiv:2307.15043*, 2023.
- [5] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, ““Do Anything Now”: Characterizing in-the-wild jailbreak prompts on large language models,” *arXiv preprint arXiv:2308.03825*, 2023.
- [6] A. Wei, A. Haghtalab, and J. Steinhardt, “Jailbroken: How does LLM safety training fail?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [7] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks, “HarmBench: A standardized evaluation framework for automated red teaming and robust refusal,” *arXiv preprint arXiv:2402.04249*, 2024.

- [8] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Schwag, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramer, H. Hassani, and E. Wong, “JailbreakBench: An open robustness benchmark for jailbreaking large language models,” in *NeurIPS Datasets and Benchmarks Track*, 2024.
- [9] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, “Llama Guard: LLM-based input-output safeguard for human-AI conversations,” *arXiv preprint arXiv:2312.06674*, 2023.
- [10] G. Alon and M. Kamfonas, “Detecting language model attacks with perplexity measures,” *arXiv preprint arXiv:2308.14132*, 2023.
- [11] N. Jain, A. Svc, A. Debenedetti, et al., “Baseline defenses for adversarial attacks against aligned language models,” *arXiv preprint arXiv:2309.00614*, 2023.
- [12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [13] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.
- [14] T. Gao, X. Yao, and D. Chen, “SimCSE: Simple contrastive learning of sentence embeddings,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [15] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS 30)*, 2017.
- [16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [17] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. CRC Press, 1994.
- [18] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: a practical and powerful approach to multiple testing,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995.
- [19] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” 2024. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>